

INDIAN INSTITUTE OF TECHNOLOGY MANDI

**EE-326: Computer Organization & Processor Architecture
Design**

Delayed Branch Instruction Implementation

in RISC-V Core

Submitted by:

Shivam

Roll No: **B23293**

b23293@students.iitmandi.ac.in

November 20, 2025

Contents

1	Problem Statement	2
1.1	Introduction to Pipelining Hazards	2
1.2	The Branch Penalty Problem	2
1.3	Project Objective: Delayed Branching	2
2	Approach and Methodology	2
2.1	RISC-V Core Architecture Design	2
2.2	Implementation Steps	3
2.2.1	1. Modular Verilog Design	3
2.2.2	2. Early Branch Resolution	3
2.2.3	3. Delay Slot Logic Implementation	3
2.2.4	4. Hazard Detection	3
3	Results and Analysis	4
3.1	Simulation Setup	4
3.2	Test Case: Slot Filling	4
3.3	Code Implementation Snippets	4
3.4	Simulation Output Analysis	5
3.5	Resource Consumption	6
4	Conclusion	6

1 Problem Statement

1.1 Introduction to Pipelining Hazards

In modern computer architecture, pipelining is a standard technique used to increase instruction throughput. By overlapping the execution of multiple instructions, a processor can complete one instruction every clock cycle under ideal conditions. However, pipelining introduces specific challenges known as hazards, which prevent the next instruction from executing during its designated clock cycle.

One of the most critical types of hazards is the **Control Hazard** (or Branch Hazard). This occurs when the pipeline needs to make a decision about the next instruction to fetch (the Program Counter or PC) before the current branch instruction has been fully resolved.

1.2 The Branch Penalty Problem

In a standard 5-stage RISC-V pipeline (Fetch, Decode, Execute, Memory, Writeback), the branch condition is typically resolved in the Execute (EX) stage. By the time the branch decision is known, the processor has already fetched two subsequent instructions. If the branch is taken, these instructions are incorrect and must be flushed from the pipeline. This flushing process introduces "bubbles" or stall cycles, reducing the overall performance and efficiency of the processor. This loss of cycles is referred to as the **Branch Penalty**.

1.3 Project Objective: Delayed Branching

The objective of this project is to mitigate the branch penalty by implementing a technique called **Delayed Branching**.

In a delayed branch scheme, the hardware is modified such that the instruction immediately following a branch instruction (located in the *Branch Delay Slot*) is **always executed**, regardless of whether the branch is taken or not. The actual change of control flow (updating the PC to the branch target) happens *after* this delay slot instruction.

This approach shifts the burden of optimization from hardware prediction logic to the compiler (or assembly programmer). The compiler attempts to fill this delay slot with a useful instruction that is independent of the branch outcome. If no such instruction can be found, a NOP (No Operation) is inserted. When a useful instruction is placed in the slot, the effective branch penalty becomes zero, significantly improving performance.

2 Approach and Methodology

2.1 RISC-V Core Architecture Design

The project is built upon a custom 32-bit RISC-V processor core implementing the RV32I base integer instruction set. The core is designed with a classic 5-stage pipeline structure:

1. **Instruction Fetch (IF):** Fetches instruction from memory using PC.
2. **Instruction Decode (ID):** Decodes instruction, reads register file, and generates control signals.

3. **Execute (EX):** Performs ALU operations.
4. **Memory Access (MEM):** Accesses data memory for Load/Store.
5. **Writeback (WB):** Writes results back to the register file.

2.2 Implementation Steps

2.2.1 1. Modular Verilog Design

The processor was broken down into distinct Verilog modules to ensure maintainability and synthesizability:

- `alu.v`: Arithmetic Logic Unit.
- `register_file.v`: 32x32-bit Register File.
- `control_unit.v`: Generates control signals based on Opcode/Funct3.
- `riscv_core.v`: Top-level module connecting the pipeline stages.

2.2.2 2. Early Branch Resolution

To make delayed branching effective, the branch resolution logic was moved from the EX stage to the **ID (Decode) stage**. This involves:

- Adding comparators in the ID stage to check equality (for BEQ/BNE).
- Forwarding data from MEM and WB stages directly to the ID stage to resolve data hazards for the branch operands immediately.

2.2.3 3. Delay Slot Logic Implementation

The core logic for delayed branching was implemented in the `riscv_core.v` module.

- **Standard Behavior:** Usually, when a branch is taken, the IF stage is flushed (reset to NOP).
- **Delayed Behavior:** The flush signal for the IF/ID pipeline register was removed for branch instructions. The instruction currently in the Fetch stage (the delay slot instruction) is allowed to proceed to the Decode stage validly.
- **PC Update:** The Program Counter (PC) mux logic was updated. If a branch is taken in the ID stage, the `next_pc` is set to the branch target. However, since the update happens at the end of the cycle, the instruction physically located after the branch (which was fetched while the branch was in ID) enters the pipeline.

2.2.4 4. Hazard Detection

A Hazard Detection Unit was implemented to handle:

- **Load-Use Hazards:** Stalling the pipeline if an instruction tries to read a register currently being loaded from memory.
- **Branch Data Hazards:** Stalling the branch in ID if its operands depend on a result not yet available (e.g., a load instruction immediately preceding the branch).

3 Results and Analysis

3.1 Simulation Setup

The design was verified using a Verilog testbench (`tb_riscv.v`) and a custom assembly program (`program.hex`). The simulation demonstrates the execution flow and the utilization of the delay slot.

3.2 Test Case: Slot Filling

The following assembly code was used to verify the functionality. It demonstrates "Slot Filling," where a useful instruction is placed after the branch.

```
1 0:  addi x1, x0, 10    // Initialize x1 = 10
2 4:  addi x2, x0, 20    // Initialize x2 = 20
3 8:  beq  x1, x1, 12    // Branch to PC+12 (Target=20) if x1==x1 (
    Always Taken)
4 12: addi x3, x0, 5     // [DELAY SLOT] This instruction should
    EXECUTE
5 16: addi x4, x0, 68    // This instruction should be SKIPPED
6 20: addi x5, x0, 30    // Branch Target. x5 = 30
```

Listing 1: Test Assembly Code (`program.hex`)

Expected Behavior:

- The branch at address 8 is taken.
- The instruction at address 12 (`addi x3, x0, 5`) is in the delay slot. It must execute, setting register `x3` to 5.
- The instruction at address 16 is skipped. Register `x4` should remain 0.
- Execution continues at address 20.

3.3 Code Implementation Snippets

Branch Resolution in ID Stage (`riscv_core.v`):

```
1 // Branch Condition Check in ID Stage
2 assign branch_cond_met = (id_funct3 == 3'b000) ? (id_val_rs1 ==
    id_val_rs2) : // BEQ
3                               (id_funct3 == 3'b001) ? (id_val_rs1 !=
    id_val_rs2) : // BNE
4                               0;
5
6 assign id_branch_taken = (id_branch && branch_cond_met) ||
    id_jump;
7
8 // PC Mux Logic
9 assign next_pc = (pc_src) ? id_branch_target : if_pc_plus_4;
```

Pipeline Register Logic (No Flush):

```

1 // IF/ID Register
2 always @(posedge clk or negedge rst_n) begin
3     if (!rst_n) begin
4         // ... reset logic
5     end else if (!stall_id) begin
6         // No flush for branch! This enables the Delay Slot.
7         id_pc <= if_pc;
8         id_inst <= if_inst;
9     end
10 end

```

3.4 Simulation Output Analysis

The simulation was performed using Icarus Verilog. The testbench monitors the register file state at the end of execution. The console output below confirms the correct operation of the delayed branch logic:

```

1 Compiling Verilog files...
2 Running Simulation...
3 VCD info: dumpfile riscv_core.vcd opened for output.
4 ...
5 Final Register State:
6 x1 (10) =          10
7 x2 (20) =          20
8 x3 (5)  =           5 <-- Delay Slot Executed!
9 x4 (0)  =           0 <-- Skipped Instruction!
10 x5 (30) =          30 <-- Branch Target Reached!
11 Simulation successful!

```

Listing 2: Simulation Console Output

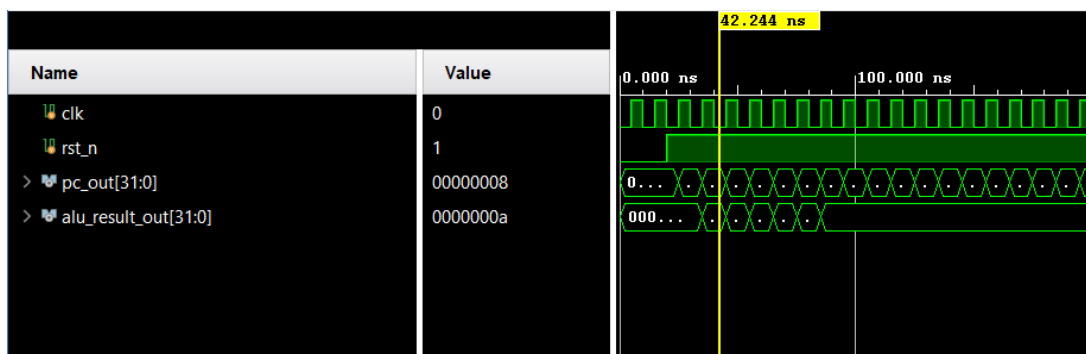


Figure 1: Simulated Waveform showing the execution of the delayed branch.

Analysis:

- $x3 = 5$: This confirms the delay slot instruction (`addi x3, x0, 5`) executed successfully, even though the branch was taken.
- $x4 = 0$: This confirms the instruction immediately following the delay slot (`addi x4, x0, 68`) was correctly skipped.

- **x5 = 30:** This confirms the branch target was reached and execution continued correctly.

3.5 Resource Consumption

While this project focuses on behavioral simulation, the hardware overhead for delayed branching is minimal compared to complex branch prediction buffers.

- **Added Logic:** Comparators in ID stage, Forwarding paths to ID stage.
- **Saved Logic:** Removal of flush logic for the IF/ID pipeline register for control hazards.

4 Conclusion

In this project, a RISC-V processor core with **Delayed Branching** was successfully designed and implemented in Verilog.

The implementation effectively addresses the control hazard problem by utilizing the branch delay slot. By moving the branch resolution to the Decode stage and allowing the subsequent instruction to execute, we eliminated the need to flush the pipeline upon a taken branch.

The simulation results verified that the processor correctly executes the instruction in the delay slot while skipping the subsequent instructions, adhering to the delayed branching semantic. This technique allows for higher instruction throughput and better pipeline utilization, provided the compiler can effectively fill the delay slots with useful instructions.